

Overview of Statistical Learning and Modeling – 36-600

Lab 6R: Principal Components Analysis

Due Friday, October 3, 11:59pm

Trishla Nair

To answer the questions below, it may again help you to refer to the ISLR book (the James et al. book listed on the syllabus), specifically Sections 10.2 and 10.4, as well as your previous labs.

Data

We'll begin by importing a dataset related to breast cancer:

```
df      <- read.csv("https://raw.githubusercontent.com/FSKoerner/F25-36600-data/refs/heads/main/breastcancer.csv")
response <- df[, 1] # B for benign, M for malignant
df      <- df[, -1]
```

These data reside on [Kaggle](#). They provide information on breast cancer tumors (read: features extracted from actual images of cells!) for 569 people in which malignancy was suspected. The data are marked by *extreme* multicollinearity and redundancy; for instance, there are columns `Mean.Radius`, `Mean.Perimeter`, and `Mean.Area`, which give almost the exact same statistical information. Also, the first ten columns are paired with the last ten columns: `Mean.Radius` with `Worst.Radius`, etc. If, for instance, 100 tumor cells are examined for a given person, the `Mean.Radius` will be the average tumor radius for all 100 data, and the `Worst.Radius` will be the average tumor radius for the three largest tumors. So, it stands to reason, the `Mean/Worst` variable pairs are going to be correlated variable pairs, potentially quite strongly.

In short, this is a dataset put together by someone with no experience in experimental design. But also, in short, this is a dataset where the data would appear to reside in a subspace that is going to be smaller than the original 20 dimensions. What can PCA tell us in this regard?

Question 1

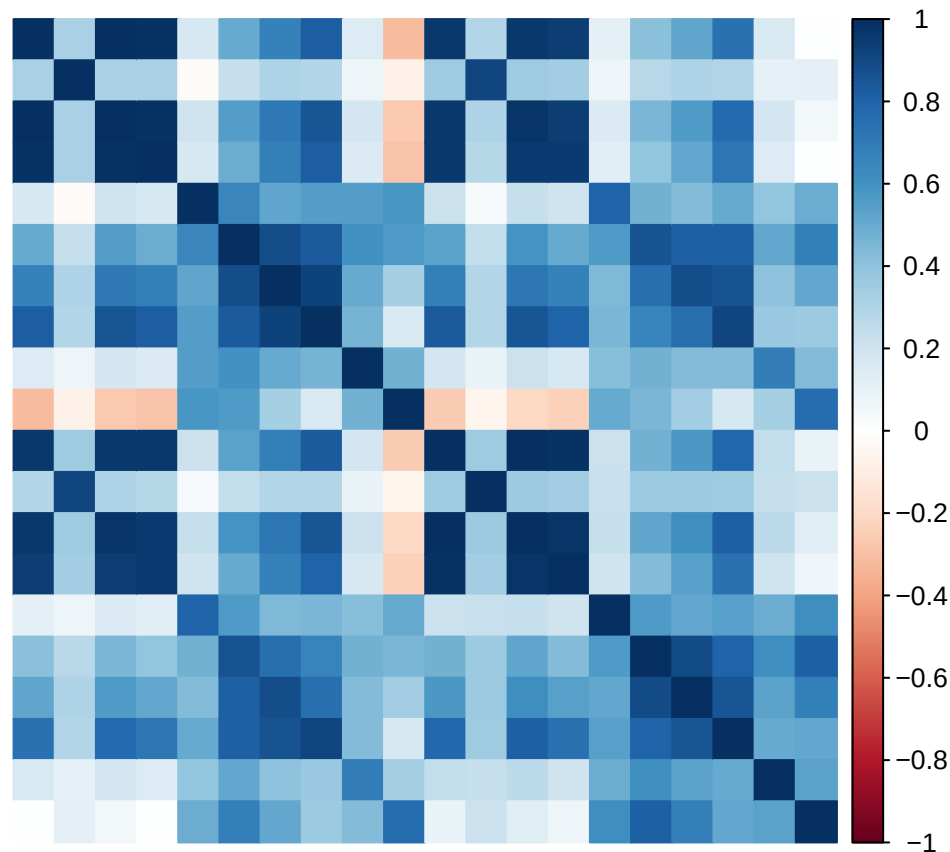
Construct a `corrplot` for `df`. Do the data appear to be correlated? (Just answer that question to yourself, no need to write anything down.) Note that adding the variable names leads to a “squashed” correlation plot. Since here you are simply visually assessing whether the data are correlated, without having to identify particular variables, I would suggest adding the argument `tl.pos = "n"` to the `corrplot()` call. (*Note*: how did I figure out that this was an option? I looked in the documentation, didn't immediately see how to remove variable names, and Googled. Of course, the answer is in the documentation, but StackOverflow had the answer too. Never stop Googling!)

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
```

```
## corrplot 0.95 loaded
```

```
corrplot(cor(df), method = "color", tl.pos = "n")
```



Question 2

Perform PCA on these data. (Be sure to look at the documentation, as there is one particular argument to `prcomp()` that you'll want to set!) Construct a plot showing the proportion of variance explained. (See page 403 of ISLR [1st ed.] to see an example of a plot constructed using the `cumsum()` function. Try to emulate that one, but use `ggplot()` instead of base-R plotting functions.) How many PCs would you retain, if you were to make a choice?

```
pca.out <- prcomp(df[, -7], scale = TRUE)
v <- pca.out$sdev^2
c <- round(cumsum(v / sum(v)), 3)

pca.df <- data.frame(PC = 1:length(v), PropVar = v, CumPropVar = c)

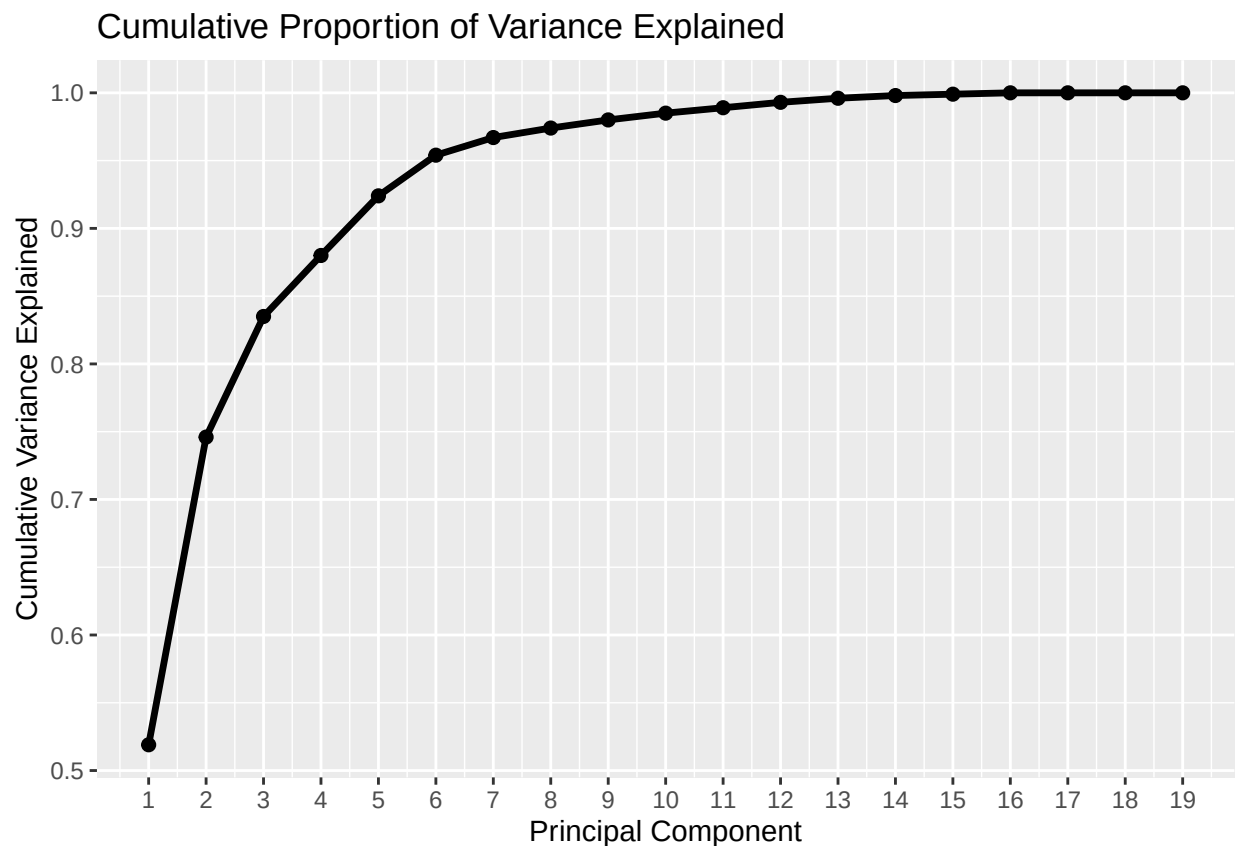
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
# Cumulative variance explained plot
ggplot(pca.df, aes(x = PC, y = CumPropVar)) +
  geom_line(color = "black", size = 1.2) +
```

```
geom_point(color = "black", size = 2) +
scale_x_continuous(breaks = 1:length(v)) +
labs(
  title = "Cumulative Proportion of Variance Explained",
  x = "Principal Component",
  y = "Cumulative Variance Explained"
)
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



Based on the plot, I would use 4 components.

Question 3

Show the first column of the rotation matrix output by `prcomp()`. (Recall that you can access the first column of a matrix or data frame by appending `[, 1]` to the name of the matrix or data frame.) This shows the relative weighting of the contribution of each original variable to the first PC. (Don't worry about any minus signs.) Make a mental note: do many/most of the variables contribute to PC1, or just a couple/few? As far as interpretation: recall that if you square every number you observe and add them all together,

you get 1. For those of you comfortable with linear algebra, what you are observing is a unit-length vector defined in the data's native space.

```
pca.out$rotation[, 1]
```

```
##          Mean.Radius      Mean.Texture      Mean.Perimeter
##          -0.26529340      -0.12676258      -0.27378091
##          Mean.Area      Mean.Smoothness      Mean.Compactness
##          -0.26308725      -0.16392386      -0.26858006
##      Mean.ConcavePoints      Mean.Symmetry      Mean.FractalDimension
##          -0.30119662      -0.15184931      -0.05069701
##          Worst.Radius      Worst.Texture      Worst.Perimeter
##          -0.27823492      -0.13873755      -0.28641011
##          Worst.Area      Worst.Smoothness      Worst.Compactness
##          -0.27043343      -0.16511840      -0.25151145
##          Worst.Concavity      Worst.ConcavePoints      Worst.Symmetry
##          -0.26662057      -0.30143983      -0.16146978
##      Worst.FractalDimension
##          -0.15378824
```

Question 4

Repeat what you did in Question 3 for PCs 2-6. (You could do this compactly by referring to `rotation[, 2:6]`.) Do particular variables map to these? (Again, just make mental notes. As always, feel free to call one of us over if you need help with interpretation.)

```
pca.out$rotation[, 2:6]
```

```
##          PC2      PC3      PC4      PC5
##      Mean.Radius      0.25175795 -0.09650689  0.0065570230 -0.0006953182
##      Mean.Texture      0.08471838  0.65152387  0.0475698804  0.1827517536
##      Mean.Perimeter      0.23021574 -0.09640710 -0.0039625830 -0.0044477844
##      Mean.Area      0.25112774 -0.10455901 -0.0008615685  0.0304273584
##      Mean.Smoothness      -0.27290246 -0.20485089  0.0245192840  0.5939156411
##      Mean.Compactness      -0.19387226 -0.05463600 -0.1251939401 -0.0514844088
##      Mean.ConcavePoints      0.01728312 -0.11759901 -0.0331520609  0.0993783405
##      Mean.Symmetry      -0.24980366 -0.08761768  0.6443635562 -0.0255339277
##      Mean.FractalDimension      -0.42155764 -0.03123838 -0.2121878476  0.0413114195
##      Worst.Radius      0.21871438 -0.05837141  0.0197622004  0.0127260860
##      Worst.Texture      0.04026059  0.66155906  0.0687024291  0.1769386442
##      Worst.Perimeter      0.19668087 -0.05517124  0.0036304980 -0.0142775512
##      Worst.Area      0.22075362 -0.06474338  0.0095155074  0.0440515286
##      Worst.Smoothness      -0.28207239 -0.02553602 -0.0431740440  0.4862685363
##      Worst.Compactness      -0.20821507  0.09847446 -0.1784745836 -0.3181000859
##      Worst.Concavity      -0.13615372  0.07217207 -0.1908556578 -0.2844188002
##      Worst.ConcavePoints      -0.03469732 -0.03462521 -0.0737139951 -0.0597043209
##      Worst.Symmetry      -0.23369523  0.04670050  0.5978291732 -0.2943097997
##      Worst.FractalDimension      -0.35602908  0.10631938 -0.2861555363 -0.2360070101
##          PC6
##      Mean.Radius      0.03461021
##      Mean.Texture      0.24566411
##      Mean.Perimeter      0.06356704
```

```
## Mean.Area          0.07653053
## Mean.Smoothness    0.01680116
## Mean.Compactness   0.31966076
## Mean.ConcavePoints 0.19807836
## Mean.Symmetry      0.38567710
## Mean.FractalDimension 0.41094005
## Worst.Radius       -0.07305599
## Worst.Texture      -0.05263447
## Worst.Perimeter    -0.03996399
## Worst.Area         -0.03633791
## Worst.Smoothness   -0.52625475
## Worst.Compactness  -0.11945635
## Worst.Concavity    -0.10364558
## Worst.ConcavePoints -0.13989274
## Worst.Symmetry     -0.36220070
## Worst.FractalDimension -0.07412137
```

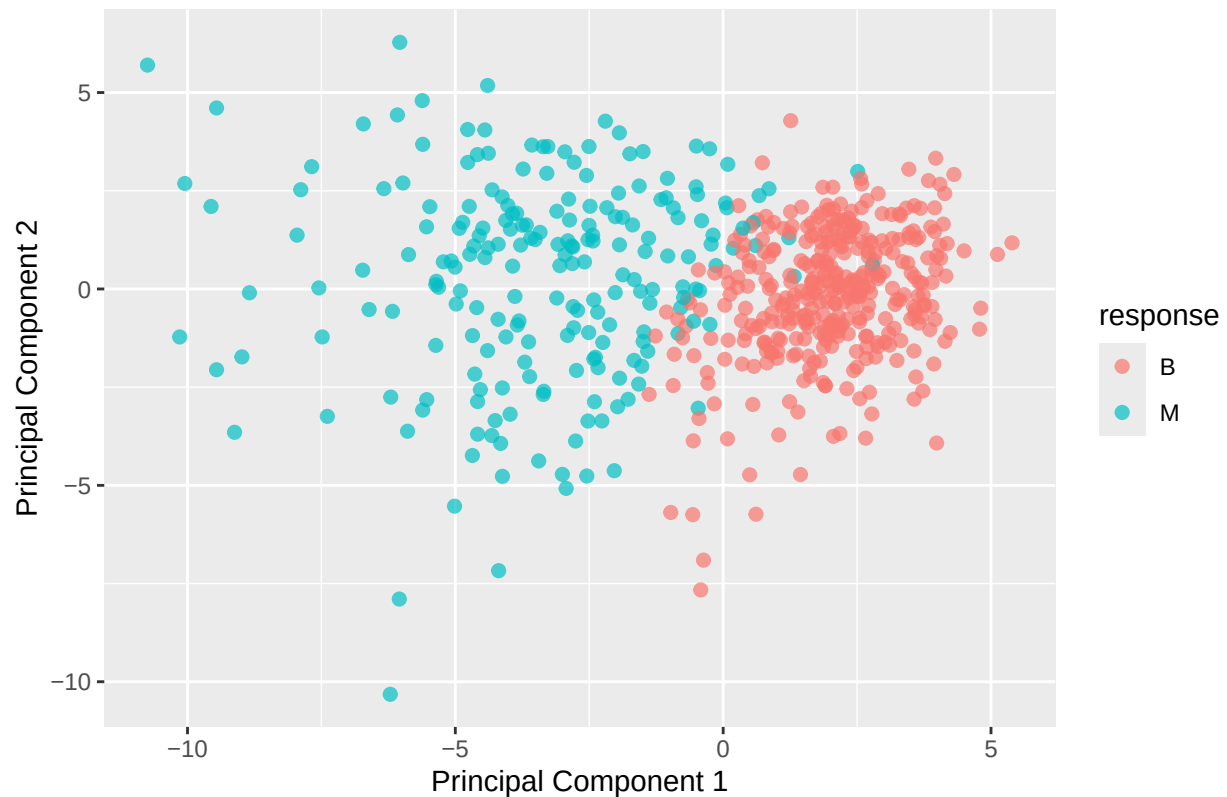
Question 5

Visualize, via scatterplot, the coordinates of the data along the first and second PC axes. This information is kept in the first and second columns of the `x` matrix output by `prcomp()`, accessible by `x[, 1:2]`. For fun, color the data using values of the `response` variable that we set aside previously when we input the data. You can do this most simply by adding `color = response` as an argument in the call to `aes()`. `ggplot2` will figure out how to use the levels of the variable to determine colors (as if by magic). Consider: does it look like benign and malignant tumors separate well in this PC subspace?

```
pc.scores <- as.data.frame(pca.out$x[, 1:2])
colnames(pc.scores) <- c("PC1", "PC2")

pc.scores$response <- df$response
ggplot(pc.scores, aes(x = PC1, y = PC2, color = response)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(
    title = "PCA: Data projected onto PC1 and PC2",
    x = "Principal Component 1",
    y = "Principal Component 2"
  )
```

PCA: Data projected onto PC1 and PC2

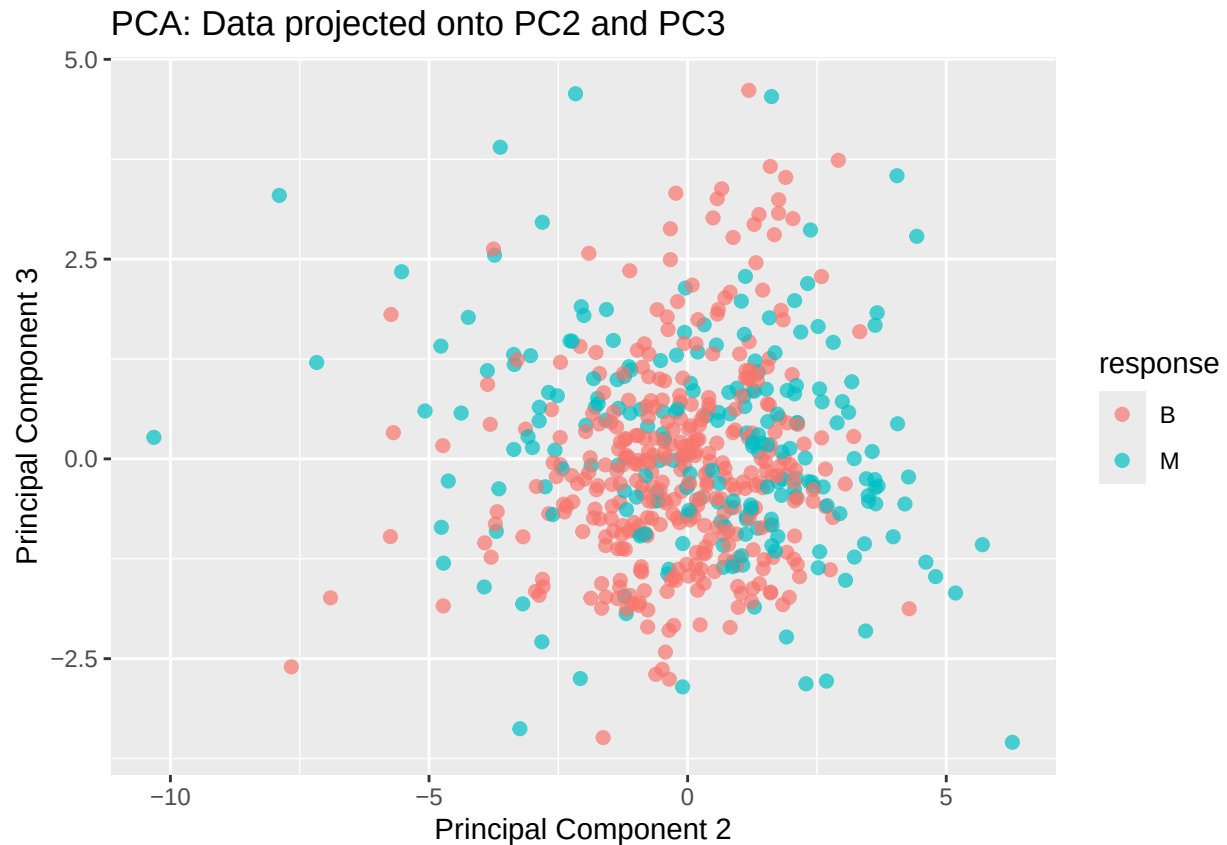


There is a part where the 2 overlap.

Question 6

Repeat Question 5, but for PC2 and PC3. Does it appear (visually) that PC3 contains information useful for classifying the two **response** classes? How about PC3 and PC4? Again, make mental notes and call us over as necessary.

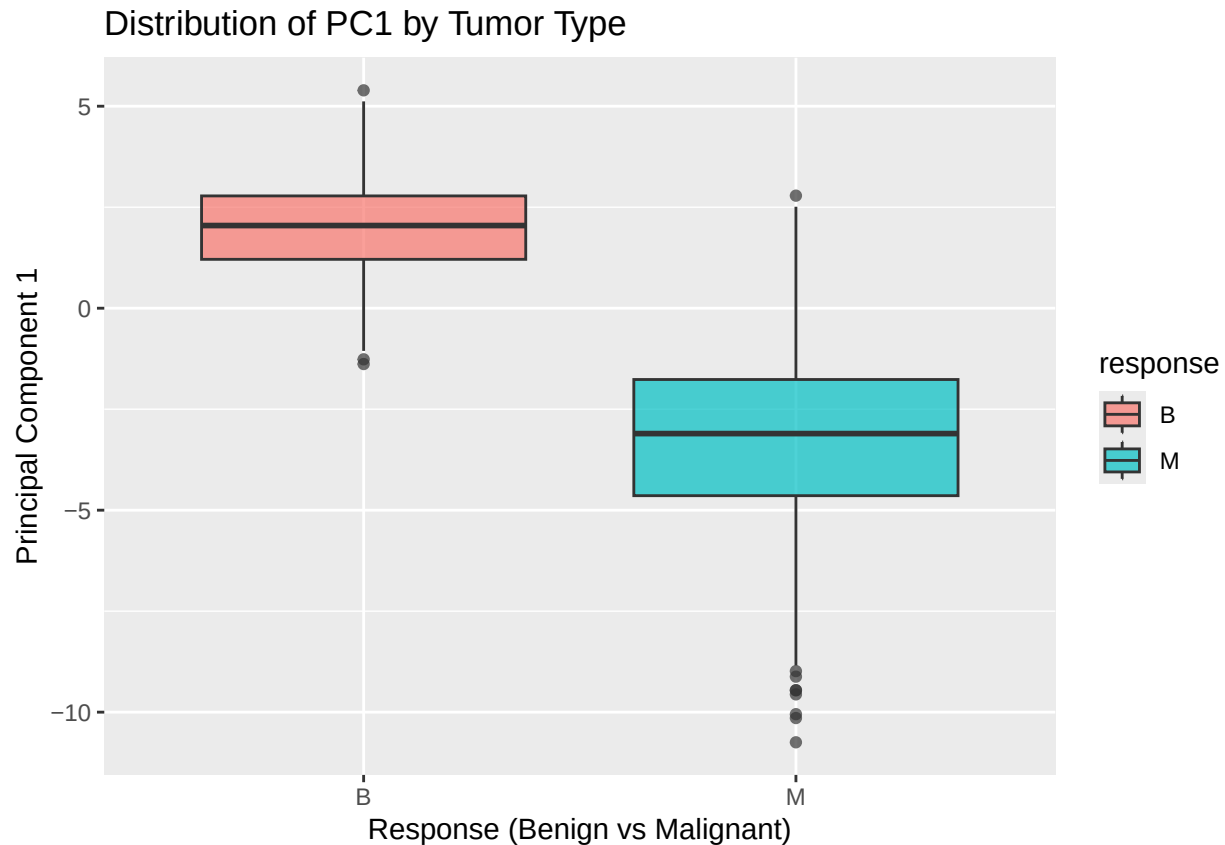
```
pc.scores <- as.data.frame(pca.out$x[, 2:3])
colnames(pc.scores) <- c("PC2", "PC3")
pc.scores$response <- df$response
ggplot(pc.scores, aes(x = PC2, y = PC3, color = response)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(
    title = "PCA: Data projected onto PC2 and PC3",
    x = "Principal Component 2",
    y = "Principal Component 3"
  )
```



Question 7

You know, given the results in Question 5, let's just visualize the data of PC1 using side-by-side box plots, based on the values of `response`. This is a good tool to review anyway. You should see that the boxes are widely separated, implying that any *classification algorithm* should do a good job predicting tumor type.

```
pc.df <- data.frame(PC1 = pca.out$x[, 1], response = response)
ggplot(pc.df, aes(x = response, y = PC1, fill = response)) +
  geom_boxplot(alpha = 0.7) +
  labs(
    title = "Distribution of PC1 by Tumor Type",
    x = "Response (Benign vs Malignant)",
    y = "Principal Component 1"
  )
```



Question 8

Let's cheat a little bit: we're going to do a logistic regression on the PCs *before* talking about logistic regression in class. The good news is, that means I give you all the code. To generate a *misclassification rate* given n PCs, you can run the following:

```
n<- 4 # don't read into this specific number - you'll change it
df<- data.frame(pca.out$x[, 1:n])
glm.out<- suppressWarnings(glm(response ~ ., data = df, family = binomial))
# Note we didn't split the data here
glm.prob <- predict(glm.out, type = "response")
glm.pred <- ifelse(glm.prob > 0.5, "M", "B")
(t
  <- table(glm.pred, response))
cat("The misclassification rate is ", round((t[1, 2] + t[2, 1])/sum(t), 3), "\n")
```

What is the misclassification rate for 1 PC? What about for your chosen number of PCs from above? (Copy the code twice, and the output will provide the answers. Make a mental note: do the extra PCs “help” or do they barely nudge the rate down at all? The plots you made above may give you an initial intuition about the answer – that intuition might be right, but it might also be flawed.) What you will see is that logistic regression on the PCs works well, but again, if your goal is prediction, you don't need to worry about multicollinearity and thus you need not compute PCs prior to doing classification. If your goal is inference, you'll still need to explain to your readers what each individual PC represents (in terms of the original variables).


```

# FILL ME IN WITH CODE
n <- 6
df<- data.frame(pca.out$x[, 1:n])
glm.out<- suppressWarnings(glm(response ~ ., data = df, family = binomial))
# Note we didn't split the data here
glm.prob <- predict(glm.out, type = "response")
glm.pred <- ifelse(glm.prob > 0.5, "M", "B")
(t      <- table(glm.pred, response))

```

```

##           response
## glm.pred  B    M
##           B 352  7
##           M   5 205

```

```

cat("The misclassification rate is ", round((t[1, 2] + t[2, 1])/sum(t), 3))

```

```

## The misclassification rate is 0.021

```

Adding more components decreases the misclassification rate.